DRAFT SKELETON — CONTAINS NO REAL RESULTS

Does Security Scaffolding Still Matter? Prompt Enrichment and Iterative Repair Across LLM Generations

Banerjee, Kumari, Ranjan, Soni
Department of Information Technology
K. J. Somaiya School of Engineering
Supervisor: Prof. Deepti Patole

Template notice. This is the paper's structure, not its findings. Every value shown as **[like this]** is a slot awaiting a number generated by `analysis/rq_analysis.py` from the run database. The experiment described in Section 4 has not yet been executed at full scale.

Abstract

Large language models are now routinely used to generate application code, and a well-established line of work (2022–2024) showed that two inference-time interventions measurably improve the security of that code: warning the model about relevant weaknesses *before* generation (prompt enrichment), and feeding static-analysis findings back for correction *after* it (scanner-guided repair). Those results were obtained on the models of that era. We ask whether they still hold. We evaluate **[N]** models spanning **[K]** capability tiers on **[50]** security-sensitive Python tasks under three conditions (unaided, enriched, enriched + repaired), with **[3]** repetitions per cell, judging every output with two independent static analysers *and* executing it against per-task functional tests. We find that baseline vulnerability rates **[rise/fall/hold]** across tiers (**[x]**% to **[y]**%), that enrichment **[does/does not]** retain its benefit on current models (Δ**[z]** pp), and—our sharpest result—that **[r]**% of repairs a scanner certifies as clean no longer pass their functional tests, a failure mode invisible to every prior evaluation of scanner-guided repair. We release the harness, the frozen task set, and the complete run database.

1 Introduction

Pearce et al. [1] reported that roughly 40% of code generated by GitHub Copilot in security-relevant scenarios contained a weakness. The finding shaped a research programme: if models write insecure code, either teach them not to (fine-tuning, constrained decoding) or catch it at inference time (enrichment, repair). The inference-time branch is the practically important one, because it is the only branch available to the overwhelming majority of practitioners, who consume models through an API and will never touch a weight or a logit.

That branch's evidence base, however, predates the current model generation. SecuCoGen [3] demonstrated CWE-aware prompting on GPT-3.5-class models; HexaCoder [6] and SVEN [7] showed oracle-guided repair, but as training-time procedures; the Copilot replication study [8] documented that a single assistant's security behaviour drifts substantially between versions. Models have since improved markedly at producing plausible, idiomatic code unaided. Whether the scaffolding built for weaker models still earns its place is an empirical question that, to our knowledge, nobody has re-asked with a consistent pipeline across model tiers.

A second gap is sharper. Every evaluation of scanner-guided repair we are aware of scores repair by re-running the scanner: if the analyser stops complaining, the repair succeeded. That criterion cannot distinguish a genuine fix from a deletion. A model that resolves a SQL-injection finding by removing the database call satisfies the scanner completely. CODEGUARD+ [5] and CodeSecEval [4] both argue that security must be judged alongside functional correctness, and both measure the two together for *generation*. Neither measures it inside a live, multi-round repair loop, which is exactly where the incentive to silently delete functionality is strongest.

Contributions.

- A quantification of how much prompt enrichment and scanner-guided repair still change security outcomes across **[K]** model tiers, using one pipeline, one task set, and one judge (§5.1–§5.3).

- A residual-risk catalogue: the weakness categories that survive *full* scaffolding on current models (§5.4) — the practitioner-facing result.
- **Repair inflation:** the first measurement, to our knowledge, of how often scanner-certified repairs are functionally broken, measured inside the repair loop and against a never-repaired control (§5.5).
- An open harness, a frozen [50]-task set with provenance, and the full run database, such that every number here is regenerable by one command.

2 Background and Related Work

Insecure generation. *[TODO: Pearce et al.; Siddiq & Santos' SecurityEval; Yan et al. on self-generated hints. Establish the baseline rates prior work reports, which our RQ1 re-measures.]*

Prompt-level intervention. *[TODO: SecuCoGen's CWE-aware prompting; Yan et al.'s self-generated and contextualised hints, including their finding that imprecise hints can* increase *vulnerability rates — directly relevant to our RQ2 discordant-pair analysis.]*

Repair. *[TODO: HexaCoder's oracle-report-plus-hint format, which our Engine C mirrors at inference time; SVEN; Pearce et al. on zero-shot repair. Emphasise: all evaluate repair by re-scanning only.]*

Security *and* correctness. *[TODO: CODEGUARD+'s secure-pass@k; CodeSecEval. This is the lineage our RQ5 extends from generation into the repair loop.]*

What we deliberately do not do. Fine-tuning approaches (LPO/DiSCo, HexaCoder's training stage) require weight access. We restrict ourselves to what an API consumer can do, which is the population the result is for.

3 The LeBlanc Pipeline

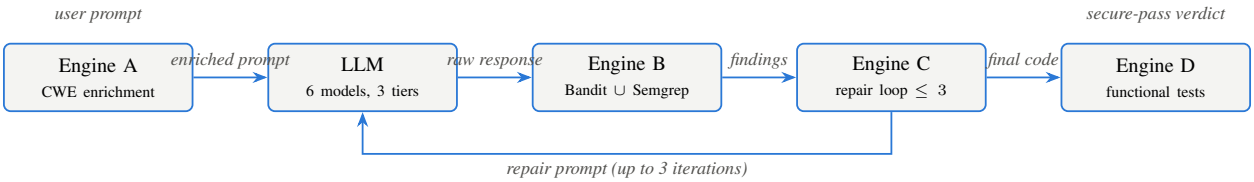

Figure 1: The LeBlanc v3 pipeline. Engine A injects CWE warnings before generation (rule-based, no LLM call); Engine B scans the generated code with two independent static analysers; Engine C feeds findings back to the *same* model for up to three repair rounds; Engine D executes the final code against per-prompt functional tests, which is what makes a "clean" verdict falsifiable (RQ5).

Engine A — enrichment. Hybrid retrieval over a curated [46]-entry CWE corpus: word-boundary lexical matching fused with TF–IDF cosine similarity, top-[5] matches injected as explicit warnings. Deterministic, offline, no model call — the intervention costs nothing at inference time, which is why the interesting question is whether it still *does* anything, not whether to deploy it.

Engine B — the judge. Bandit ($\geq$ medium severity) unioned with Semgrep against a pinned local copy of the `p/python` pack ([781] rules), deduplicated by line, CWE set, and a cross-tool equivalence map. Pinning matters for reproducibility: a live registry pack changes underneath a longitudinal experiment. Extraction and syntax failures are their own outcome class and are never counted as clean.

Engine C — repair. Findings are formatted as a numbered list with CWE context and returned to the *same* model, with an explicit instruction to preserve the public interface; the patched output is re-scanned. Capped at [3] rounds. Non-convergence is reported, not discarded.

Engine D — the honesty layer. The final code is executed against per-task `pytest` smoke tests in a sandboxed subprocess with no network, using an offline harness that fakes MySQL, LDAP, and network I/O so that database- and auth-shaped tasks remain testable. This is what makes "clean" falsifiable and what RQ5 rests on.

4 Experimental Design

Tasks. [50] Python tasks, frozen before data collection: [40] drawn from SecurityEval [2] by stratified selection (max 2 per CWE, provenance recorded per task) and [10] written by the authors. The authored subset is a deliberate hedge against benchmark contamination: SecurityEval has been public since 2022 and may appear in current training corpora, whereas the authored tasks have not. [20] tasks carry functional tests validated against hand-written secure reference solutions; exclusions are documented per task.

Models. [6] models across [3] tiers and [3] vendors. *[TODO: Table of model, vendor, tier, and the verify-at-build date. State plainly here that provider retirement of older models mid-study means tiers separate capability/size, not calendar generation — see Threats to Validity.]*

Conditions. `plain` (no intervention), `enriched` (Engine A only), `enriched_repair` (Engines A and C). [3] repetitions per cell at temperature [0.7]; the non-zero temperature is deliberate, so repetitions are genuine independent samples rather than near-duplicates.

Measures and statistics. Vulnerability rate with Wilson 95% intervals; exact McNemar on paired plain/enriched outcomes; χ^2 (Fisher for small cells) across tiers; Mann–Whitney U on iteration counts; and `vuln ~ mode * tier + category` as the headline logistic model. Decision bands separating "yes", "mixed", and "no" were fixed in the project's public documentation *before* data collection and are applied mechanically by the analysis script.

Three further precautions, each because the naive alternative would overstate what we know. First, **repetitions are nested within tasks**: three samples of one task are not three independent observations, so the logistic model uses standard errors clustered on task, and every headline rate is additionally reported with a cluster bootstrap over tasks (resampling tasks, not runs). Second, **we run one test per model**, so p-values within a family are corrected for multiple comparisons (Benjamini–Hochberg, FDR $\alpha = 0.05$); we report adjusted values. Third, every analysis choice that could move a headline number is reported as a **sensitivity range** rather than settled silently (§5.6).

5 Results

5.1 RQ1 — Baseline vulnerability by tier

[Insert analysis/output/tables/rq1.tex] [Figure: figures/rq1_baseline_vr.png]
Unaided vulnerability rates range from **[x]**% to **[y]**%. *[TODO: State whether the ordering predicted by H1 holds, and compare the absolute level against the ≈40% Pearce et al. figure.]*

5.2 RQ2 — Does enrichment still help?

[Insert analysis/output/tables/rq2.tex] [Figure: figures/rq2_enrichment_delta.png]
[TODO: Report ΔE per model with McNemar p. Report discordant pairs in both directions — cases where enrichment turned clean output vulnerable are a finding, not noise, and prior work reports the same effect for imprecise hints.]

5.3 RQ3 — Does repair converge?

[Insert analysis/output/tables/rq3.tex] [Figure: figures/rq3_convergence.png]
[TODO: Convergence rate and mean iterations per model; report the non-convergence rate prominently rather than as a footnote.]

5.4 RQ4 — What survives full scaffolding

[Figure: figures/rq4_residual_heatmap.png]
[TODO: Name the categories in the red band on current models. This is the section a practitioner reads.]

5.5 RQ5 — Repair inflation

[Insert analysis/output/tables/rq5.tex] [Figure: figures/rq5_repair_inflation.png]
Of repairs the scanner certified clean, **[r]**% fail their functional tests. The comparison that supports the causal reading is against never-repaired but equally scanner-clean code, which fails at **[b]**%; the difference attributable to repair is therefore **[d]** pp (Fisher $p =$ **[p]**).

[TODO: Be disciplined about which sentence the data licenses. If the attributable difference is small while both rates are high, the finding is "scanner-clean LLM code frequently does not work" — valuable, and a direct extension of the secure-pass@k argument — and is not *"the repair loop breaks code". Write whichever one is true.]*

[TODO: Include the qualitative coding of how repairs break functionality: interface change, stub-out, dependency drift. One worked example per mode.]

5.6 Sensitivity — does the headline survive its own assumptions?

[Insert analysis/output/tables/sensitivity.tex]
Three analysis choices could each move the baseline rate, so we report the range rather than one convenient point.

Which findings count. Our security verdict is the union of two analysers, and not every rule they fire is a weakness in the code that was requested. Excluding the rule families the false-positive audit judged inapplicable moves the baseline rate to **[a]**%; excluding findings located in the demonstration scaffolding models append (`app.run(...)`, `__main__` blocks) rather than in the requested function moves it to **[b]**%; excluding both gives **[c]**%, against a headline of **[d]**%. *[TODO: If this spread is large — in our pilot it was over 30 percentage points for one model — then the headline figure is substantially a statement about Flask boilerplate, and the conservative figure belongs in the abstract alongside it. Do not report only the larger number.]*

Unparseable output. Runs whose output did not yield compilable code are excluded from the rate denominator; a model that emitted no code produced nothing to judge. Because that failure rate differs by model (**[e]**% to **[f]**%), we also report the rate with those runs counted as clean and as vulnerable, bounding the effect of the choice at **[g]** pp.

Clustering. Naive intervals treat three repetitions of one task as three independent observations. Task-level bootstrap intervals are **[h]** pp wider on average; we quote those.

6 Threats to Validity

Static-analysis error. Our security verdict is Bandit ∪ Semgrep. A stratified **[10]**% sample was labelled TP/FP, giving a false-positive rate of **[f]**% **[(with κ = k, or: single-annotator triage, no κ)]**. *[TODO: If the FP rate is high, say so here and in the abstract, identify which rules drive it, and report rates with those rules excluded as a sensitivity analysis.]* We additionally report what fraction of findings land on scaffolding the model volunteers (`app.run` blocks) rather than on the requested function, since these inflate rates without reflecting the task.

Tiers are not calendar generations. Provider retirement of older models during the study means our tiers separate capability and size among concurrently available models. The generational claim is correspondingly weaker and stated as such.

Smoke tests are not correctness. Engine D establishes that code still does its basic job; it does not establish full functional equivalence. It can therefore only ever *underestimate* breakage — which makes RQ5 a lower bound.

Scope. Python only; [50] tasks; a 3-iteration repair cap; one enrichment strategy. [20] of [50] tasks carry functional tests, so RQ5's sample is smaller than RQ1–RQ4's and is reported at its true n.

7 Conclusion

[TODO: Practitioner guidance, stated as a decision rule: for which model tiers and which weakness categories does inference-time scaffolding still pay, and where is it ceremony? Then the honest caveat about what a scanner-clean verdict does and does not buy.]

Availability. Harness, frozen task set, analysis scripts, and the complete run database: *[TODO: repository URL]*.

References

[1] H. Pearce et al., "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," *IEEE S&P*, 2022.

[2] M. L. Siddiq and J. C. S. Santos, "SecurityEval Dataset," *MSR4P&S*, 2022.

[3] J. Wang et al., "Enhancing Large Language Models for Secure Code Generation: A Dataset-driven Study on Vulnerability Mitigation," 2024.

[4] J. Wang et al., "Is Your AI-Generated Code Really Safe? Evaluating LLMs on Secure Code Generation with CodeSecEval," 2024.

[5] Y. Fu et al., "Constrained Decoding for Secure Code Generation," 2024.

[6] H. Hajipour et al., "HexaCoder: Secure Code Generation via Oracle-Guided Synthetic Training Data," 2024.

[7] J. He and M. Vechev, "Large Language Models for Code: Security Hardening and Adversarial Testing," *CCS*, 2023.

[8] V. Majdinasab et al., "Assessing the Security of GitHub Copilot's Generated Code — A Targeted Replication Study," 2023.

[9] H. Yan et al., "Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation," 2025.

[10] Q. Zhu et al., "DomainEval: An Auto-Constructed Benchmark for Multi-Domain Code Generation," 2024.